AEM 6.5 Component Development

Complete Guide — From Basics to Full Flow

1. What Files Do You Need?

■ MUST Have (Minimum Required)

File	Purpose
.content.xml	Registers the component in AEM JCR
mycomponent.html	Renders the component on the page

That's it. A component will work with just these two files.

■■ Add ONLY If You Need It

File	When You Need It
_cq_dialog/.content.xml	Authors need to edit text/images via dialog
Sling Model (Java)	Backend logic, API calls, or complex data
clientlibs/	Component needs its own CSS or JS
_cq_editConfig	Custom drag & drop or in-place editing
_cq_design_dialog	Template-level settings for the component

■ You Do NOT Need

• JSP files — HTL (.html) is enough, JSP is old

• Two .content.xml in the same folder — one per folder only

• Sling Model — if component just displays simple text, HTL reads ${properties.title} directly

• sling:resourceSuperType — only needed if extending another component

• Core Components dependency — only if extending Core Components

2. How to Create a Component in AEM 6.5 (CRXDE Lite)

Step 1 — Open CRXDE Lite

```
http://localhost:4502/crx/de/index.jsp
```

Step 2 — Navigate to your components folder

```
/apps/myproject/components/content
```

Step 3 — Create the Component Node

```
Right click content folder → Create Node
Name: mycomponent
Type: cq:Component
```

Step 4 — Add Properties to Component Node

```
jcr:title       = My Component
jcr:description = My first component
componentGroup  = My Project
```

Step 5 — Create .content.xml

```
Right click mycomponent → Create File → .content.xml
```

Step 6 — Create mycomponent.html

```
Right click mycomponent → Create File → mycomponent.html
```

Step 7 — Save All

```
Ctrl + S
```

3. File Contents

.content.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<jcr:root xmlns:jcr="http://www.jcp.org/jcr/1.0"
          xmlns:cq="http://www.day.com/jcr/cq/1.0"
          xmlns:sling="http://sling.apache.org/jcr/sling/1.0"
    jcr:primaryType="cq:Component"
    jcr:title="My Component"
    jcr:description="My first component"
    componentGroup="My Project"/>
```

mycomponent.html (HTL)

```
<sly data-sly-use.model="com.myproject.core.models.MyModel"/>
<div class="my-component">
    <h2>${model.title}</h2>
    <p>${model.description}</p>
</div>
```

MyModel.java (Sling Model)

```
@Model(
    adaptables = Resource.class,
    defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL
```

```
)
public class MyModel {

    @ValueMapValue
    private String title;

    @ValueMapValue
    private String description;

    public String getTitle()       { return title; }
    public String getDescription() { return description; }
}
```

_cq_dialog/.content.xml

```
<jcr:root xmlns:jcr="http://www.jcp.org/jcr/1.0"
          xmlns:nt="http://www.jcp.org/jcr/nt/1.0"
    jcr:primaryType="nt:unstructured"
    jcr:title="My Component"
    sling:resourceType="cq/gui/components/authoring/dialog">
  <content sling:resourceType=
      "granite/ui/components/coral/foundation/container">
    <items>
      <tabs sling:resourceType=
          "granite/ui/components/coral/foundation/tabs">
        <items>
          <properties jcr:title="Properties"
              sling:resourceType=
              "granite/ui/components/coral/foundation/container">
            <items>
              <title
                  sling:resourceType=
                  "granite/ui/components/coral/foundation/form/textfield"
                  fieldLabel="Title"
                  name="./title"/>
            </items>
          </properties>
        </items>
      </tabs>
    </items>
  </content>
</jcr:root>
```

4. The Actual Flow — How All 4 Files Work Together

Step	What Happens
1. Author fills Dialog (_cq_dialog)	Author types "Hello World" in Title field Clicks Save
2. Data saved to JCR	/content/myproject/.../mycomponent title = "Hello World" description = "This is my component"

3. Page requested	`Browser hits the page URL`
	`AEM finds component node → reads sling:resourceType`
	`Goes to mycomponent.html`

4. HTL calls Sling Model (mycomponent.html)	`data-sly-use.model="...MyModel"` `"Hey, go run MyModel.java"`
5. Sling Model reads JCR (MyModel.java)	`@ValueMapValue reads title & description` `from the JCR node automatically`
6. HTL renders output	`${model.title} → "Hello World"` `${model.description} → rendered on page`

7. Browser gets HTML	`<div class="my-component">`
	`  <h2>Hello World</h2>`
	`</div>`

Where does .content.xml fit?

.content.xml is not part of the runtime flow. It works at registration time — it tells AEM this folder is a cq:Component, gives it a title and group, and makes it appear in the component browser so authors can drag it onto a page. Without it, AEM treats the folder as just a random folder and nothing works.

5. One-Line Summary of Each File

File	Role	One Line Summary
.content.xml	Registration	ID card — tells AEM this folder is a component
_cq_dialog	Input	The form authors use to enter data
MyModel.java	Processing	Reads data from JCR and prepares it for HTL
mycomponent.html	Output	Renders the final HTML on the page